Retrieval-Based Adaptation of Time Series Foundation Models

Experiment Guideline

Gaspard Berthelier

7 August 2026

Contents

1 Time Series Forecasting

Let $\mathcal{U} = \{0, \dots, U-1\}$ be a set of U users or channels and $t \in \{0, \dots, T-1\}$ the time index for the T measured dates. For channel $u \in \mathcal{U}$, the univariate series is $z_0^{(u)}, \dots, z_{T-1}^{(u)}$. We consider the setting of a fixed lookback length $L \geq 1$ and forecast horizon $H \geq 1$. Given a valid *query date* s, the lookback and future horizon are

$$X_{s,u} = z^{(u)}_{(s-L,s]} = (z^{(u)}_{s-L+1}, \dots, z^{(u)}_s) \in \mathbb{R}^L, \tag{1}$$

$$Y_{s,u} = z^{(u)}_{(s,s+H]} = (z^{(u)}_{s+1}, \dots, z^{(u)}_{s+H}) \in \mathbb{R}^H. \tag{2}$$

Thus the lookback length L represents the number of observed values at a given time and the horizon H is the number of target values to predict.

A forecasting model is a parameterized map

$$F_\theta : \mathbb{R}^L \longrightarrow \mathbb{R}^H, \qquad X_{s,u} \longmapsto \widehat{Y}_{s,u}.$$

Here θ denotes the model parameters.

Certain families of models allow the use of optional covariates. Let $C_{s,u}$ denote a set of covariate windows of size $L + H$ (horizon included). Whe using covariates, the prediction becomes $\widehat{Y}^{\text{cov}}_{s,u} = F_\theta(X_{s,u}; C_{s,u})$.

For any $x = (x_1, \dots, x_n) \in \mathbb{R}^n$, define

$$\mu_x = \frac{1}{n}\sum_{k=1}^{n} x_k, \qquad \sigma_x = \text{sd}(x) = \left(\frac{1}{n}\sum_{k=1}^{n}(x_k - \mu_x)^2\right)^{1/2}, \qquad \varepsilon = 10^{-8}.$$

For one sample, consisting of windows $(X, Y, \widehat{Y})$, the principal losses are

$$\text{MSE}(\widehat{Y}, Y) = \frac{1}{H}\sum_{h=1}^{H}(\widehat{Y}_h - Y_h)^2, \qquad \text{nMSE}(\widehat{Y}, Y \mid X) = \frac{1}{H}\sum_{h=1}^{H}\left(\frac{\widehat{Y}_h - Y_h}{\max(\sigma_X, \varepsilon)}\right)^2.$$

2 Foundation Models

A time series Foundation Model (FM) is pretrained over many tasks and datasets, and directly reused on unseen series without updating its parameters. Indeed, their extensive training allows them to perform In-Context Learning (ICL), which means the model has learned to detect patterns within the input, as if it were a dataset on which it was fine-tuned. A few examples of FMs are provided below.

Chronos-2. Chronos-2 is a 120M-parameter encoder-only foundation model whose group attention supports univariate, multivariate, and covariate-aware zero-shot forecasting (Ansari et al., 2025). In this project, we notably use context mode `future_included`: each covariate contributes its lookback as well as its observed horizon. The active code name is `chronos2`.

TabPFN-TS. TabPFN-TS casts forecasting as tabular regression with time, seasonal, frequency, and optional covariate features, then performs ICL with a pretrained TabPFN regressor (Hoo et al., 2025). The project wrapper uses the `tabpfn-v2.5-regressor-v2.5_default.ckpt` checkpoint, automatically adds periods 24 and 168 when L permits, and exposes the code name `tabpfnts`.

TS-ICL. TS-ICL is a 27M-parameter time-indexed encoder–regressor Transformer for forecasting and imputation on regular or irregular timestamps (Le Naour et al., 2026). It is part of the literature scope, but `ts_icl` is deliberately rejected by the current launcher because no wrapper is registered.

Chronos-Bolt. The `chronos-bolt` wrapper supplies the frozen 205M-parameter Chronos-Bolt-base backbone for the Cross-RAG comparison. It is univariate and ignores retrieved covariates; neighbour targets and forecasts remain available to the adapter (Ansari et al., 2024; Amazon Science, 2026). Source-faithful local model modules load the released base weights from `weights/chronos-bolt-base` and the sole recursive `best.pth` below `weights/cross-rag`; `CHRONOS_BOLT_WEIGHTS_PATH` and `CROSSRAG_WEIGHTS_PATH` are explicit overrides. No external code checkout or Cross-RAG retraining is part of the workflow.

Table 1: Model names present in the Python registry or publication launchers. Trainable means parameters updated by this project, not total pretrained size.

Code name	Role and current status	Total parameters	Trainable here
`chronos2`	Primary screen/full backbone; frozen	120M	0
`tabpfnts`	Ultra backbone; frozen TabPFN-v2.5 regressor	$\approx$ 11M	0
`chronos-bolt`	Fixed Cross-RAG backbone; frozen	205M	0
`persistence`	Python-registry diagnostic	0	0
`linear`	Python-registry diagnostic, univariate	$H(L+1)$	$H(L+1)$
`patchtst`	Registry diagnostic; not launched for publication	466,843 / 663,091 / 1,762,603	same
`ts_icl`	Reserved shell name; not registered	27M in the paper	not run

The three PatchTST numbers in Table 1 use the code defaults at 168:24, 336:48, and 504:168, respectively. Publication experiments accept only the three frozen backbones listed first.

3 From RAG to retrieval-augmented forecasting

Retrieval-augmented generation separates a parametric model from an explicit, replaceable non-parametric memory. Classical RAG retrieves documents from a dense index and conditions a pretrained generator on them (Lewis et al., 2020). In kNN-MT, a frozen neural machine-translation model caches decoder representations k_i with next-token values v_i; at decoding state q_t, it interpolates the model and neighbour distributions (Khandelwal et al., 2021). Let $p_\theta(\cdot \mid q_t)$ be the neural next-token distribution for candidate token y_t, let $\mathcal{N}_K(q_t)$ contain the $K \geq 1$ cached keys nearest to q_t under distance d, and choose interpolation weight $\lambda \in [0,1]$ and temperature $\tau > 0$. Then

$$p(y_t \mid q_t) = (1-\lambda)p_\theta(y_t \mid q_t) + \lambda \frac{\sum_{i \in \mathcal{N}_K(q_t)} \mathbf{1}[v_i = y_t] \exp(-d(q_t, k_i)/\tau)}{\sum_{i \in \mathcal{N}_K(q_t)} \exp(-d(q_t, k_i)/\tau)}.$$

This is the direct ancestor of our question: can a frozen pretrained model be adapted cheaply by retrieving domain examples and mixing their outcomes?

Time-series work has followed several paths: retrieved sequences can become a text prompt (TimeRAG), an appended context (RAF), an input to a model trained on the target dataset (RAFT), or values fused by pretrained attention modules (TS-RAG and Cross-RAG). Our method is narrower: the datastore is target-panel history, the backbone is frozen, and simple validated forecasts or gates are tested before a neural adapter.

Table 2: Bibliography map for pretrained retrieval and time-series forecasting. This is a method map, not a cross-paper accuracy ranking.

Method	Date / venue	Retrieval object	Adaptation mechanism
RAG (Lewis et al., 2020)	2020, NeurIPS	Dense-indexed passages	Fine-tuned seq2seq generator combines parametric and non-parametric memory.
kNN-MT (Khandelwal et al., 2021)	2021, ICLR	Decoder state / next-token pairs	No retraining; interpolates neural and neighbour token distributions.
RAF (Tire et al., 2024)	2024 preprint; v2 2026	Related time-series contexts and futures	Normalises, aligns, and prepends retrieved examples to frozen TSFMs.
TimeRAG (Yang et al., 2025)	2024 preprint; ICASSP 2025	DTW-similar historical sequences	Renders references and query as an LLM forecasting prompt.
TS-RAG (Ning et al., 2025a)	2025, NeurIPS	Encoder-space context–future pairs	Pretrains projectors and an Adaptive Retrieval Mixer; no target-task tuning.
RAFT (Han et al., 2025)	2025, ICML	Correlated training inputs and following patches	Trains a shallow multiresolution forecasting MLP per target dataset.
Cross-RAG (Lee et al., 2026a)	2026, KDD MILETS	Min–max lookback/future pairs, cosine search	Pretrained query–retrieval cross-attention with query-dependent and query-independent paths.
This project	2026	Raw or transformed historical lookback/future pairs	Closed-form baselines, CatBoost gates, and TS-IFA around a frozen backbone.

4 Neighbour retrieval

4.1 Datastore and exact search

Fix a named retrieval transform ρ and distance metric δ. Their maps are

$$\phi = \phi_\rho : \mathbb{R}^L \to \mathbb{R}^{d_\rho}, \qquad d = d_\delta : \mathbb{R}^{d_\rho} \times \mathbb{R}^{d_\rho} \to \mathbb{R}_{\geq 0},$$

with every implemented ϕ_ρ and d_δ defined explicitly in Table 3. Here $d_\rho = L$ except for the encoder space; $g_\theta : \mathbb{R}^L \to \mathbb{R}^{d_{\text{enc}}}$ denotes the frozen backbone representation after the configured flattening or pooling.

For query (s, u), let $\mathcal{R}_s = \{r_{s,0} < \cdots < r_{s,n_s^{\text{store}}-1}\}$ be the ordered finite set of $n_s^{\text{store}} = |\mathcal{R}_s|$ store dates in $\{L-1, \ldots, T-H-1\}$ that pass the fixed/online, period, stride, and cap rules in Section 7. The eligible store keys and the resulting labelled datastore are

$$\mathcal{I}_s = \mathcal{R}_s \times \mathcal{U}, \qquad \mathcal{D}_s = \{(\phi(X_{r,v}), X_{r,v}, Y_{r,v}, r, v) : (r, v) \in \mathcal{I}_s\}.$$

For $K \in \{1, \ldots, |\mathcal{I}_s|\}$, let $\text{TopK}^{\min}$ return the ordered K minimizers. Exact search returns the following store keys:

$$\mathcal{N}_K(s, u) = \text{TopK}^{\min}_{(r,v)\in\mathcal{I}_s} d(\phi(X_{s,u}), \phi(X_{r,v}))\,.$$

The datastore is flattened in *user-major, date-minor* order. For $i \in \{0, \ldots, Un_s^{\text{store}} - 1\}$, a flat index decodes as

$$v = \left\lfloor \frac{i}{n_s^{\text{store}}} \right\rfloor, \qquad m = i \bmod n_s^{\text{store}}, \qquad r = r_{s,m}.$$

Downstream query payloads use the opposite, date-major ordering $(s_0, u_0), \ldots, (s_0, u_{U-1}), (s_1, u_0), \ldots$ so that whole dates form contiguous blocks for chronological splitting; $s_0 < s_1 < \cdots$ are the saved query dates and $u_k = k$ is the internal channel index.

In Table 3, $X \in \mathbb{R}^L$ is a generic lookback; in its distance rows, $a, b \in \mathbb{R}^{d_\rho}$ are two transformed lookbacks and $x_{\min} = \min_{1\leq k\leq L} X_k$, $x_{\max} = \max_{1\leq k\leq L} X_k$, $a^\circ = a - \mu_a \mathbf{1}_{d_\rho}$, $b^\circ = b - \mu_b \mathbf{1}_{d_\rho}$ are their centred forms.

Table 3: Implemented retrieval spaces and distances.

Space	$\phi(X)$	Status
Raw	X	Primary screen
Instance	$(X - \mu_X \mathbf{1}_L)/(\sigma_X + \varepsilon)$	Primary screen
Min–max	$(X - x_{\min}\mathbf{1}_L)/\max(x_{\max} - x_{\min}, \varepsilon)$	Cross-RAG profile
Fourier	$\lvert\,\text{FFT}((X - \mu_X \mathbf{1}_L)/(\sigma_X + \varepsilon))\rvert$	Optional ablation
Encoder	flattened or pooled frozen representation $g_\theta(X)$	Implemented override
Euclidean	$\lVert a - b\rVert_2$	Primary metric
Cosine	$1 - a^\top b/[\max(\lVert a\rVert_2, \varepsilon)\max(\lVert b\rVert_2, \varepsilon)]$	Cross-RAG metric
Pearson	$1 - (a^\circ)^\top b^\circ/[\max(\lVert a^\circ\rVert_2, \varepsilon)\max(\lVert b^\circ\rVert_2, \varepsilon)]$	Implemented override

4.2 Saved information and query-scale transfer

For neighbour rank $j \in \{1, \ldots, K\}$ with $(r_j, v_j) \in \mathcal{N}_K(s, u)$, abbreviate $X_j = X_{r_j,v_j}$ and $Y_j = Y_{r_j,v_j}$, and let d_j be its retrieval distance and $N_j = F_\theta(X_j)$ its frozen forecast. Also abbreviate the query as $X = X_{s,u}$ and $Y = Y_{s,u}$. Extraction saves the quantities in Table 4; it stores $E_j = Y_j - N_j$ rather than duplicating N_j, which is reconstructed downstream.

Table 4: Current extraction payload. Optional fields are generated only when their switches are enabled.

Object	Saved content
Query	$X, Y, V = F_\theta(X), C = F_\theta(X; \{X_j, Y_j\}_{j=1}^K)$
Neighbours	$X_j, Y_j, E_j = Y_j - N_j, d_j, r_j, v_j$ for $j = 1, \ldots, K$
Indices	query date/user, neighbour date/user, eligible store-date/window counts
Summaries	query and neighbour level/scale; forecast–context and residual losses
Optional	context-conditioned neighbour residuals

Retrieved values are expressed on the query scale before fitting. Write $q = (s, u)$ for the current query and let $(\mu_q, \sigma_q) = (\mu_X, \sigma_X)$ and $(\mu_j, \sigma_j) = (\mu_{X_j}, \sigma_{X_j})$. For a neighbour-scale horizon vector $Z \in \mathbb{R}^H$ or residual vector $E \in \mathbb{R}^H$, define

$$\mathcal{T}_{j\to q}(Z) = \frac{Z - \mu_j \mathbf{1}_H}{\max(\sigma_j, \varepsilon)}\sigma_q + \mu_q \mathbf{1}_H, \tag{3}$$

$$\mathcal{T}^{\mathrm{res}}_{j\to q}(E) = \frac{E}{\max(\sigma_j, \varepsilon)}\sigma_q. \tag{4}$$

All later symbols Y_j, N_j, E_j denote these query-scale values.

Distances are converted to weights independently for every query. Let $\boldsymbol{d} = (d_1, \dots, d_K)$ be the ordered retrieved-distance vector and let $j, \ell \in \{1, \dots, K\}$:

$$\widetilde{d}_j = \frac{d_j - \min_\ell d_\ell}{\max(\sigma_{\boldsymbol{d}}, \varepsilon)}, \qquad w_j = \frac{e^{-\widetilde{d}_j}}{\sum_{\ell=1}^K e^{-\widetilde{d}_\ell}}, \qquad \overline{Y}_h = \sum_{j=1}^K w_j Y_{j,h}.$$

Thus $w_j \geq 0$, $\sum_{j=1}^K w_j = 1$, and $\overline{Y} \in \mathbb{R}^H$; for $K = 1$, $w_1 = 1$.

5 Retrieval baselines

For extracted window i with query (s_i, u_i), define the target $G_i = Y_{s_i,u_i} \in \mathbb{R}^H$, vanilla forecast $V_i \in \mathbb{R}^H$, conditioned forecast $C_i \in \mathbb{R}^H$, query-scale neighbour future and forecast $(Y_{i,j}, N_{i,j}) \in \mathbb{R}^H \times \mathbb{R}^H$ for $j \in \{1, \dots, K\}$, and

$$V_i = F_\theta(X_{s_i,u_i}), \qquad C_i = F_\theta(X_{s_i,u_i}; \{X_{i,j}, Y_{i,j}\}_{j=1}^K), \qquad \overline{Y}_i = \sum_{j=1}^K w_{i,j} Y_{i,j}.$$

Each design is $(V_i, Z_{i,1}, \dots, Z_{i,p})$.

Table 5: Baseline designs; $Z_i = (Z_{i,1}, \dots, Z_{i,p})$.

Stem	Z_i	p
`cov`	(C_i)	1
`avgy`	$(\overline{Y}_i)$	1
`y`	$(Y_{i,1}, \dots, Y_{i,K})$	K
`cov_y`	$(C_i, Y_{i,1}, \dots, Y_{i,K})$	$1 + K$
`cov_avgy`	$(C_i, \overline{Y}_i)$	2
`residual`	$(Y_{i,1}, \dots, Y_{i,K}, N_{i,1}, \dots, N_{i,K})$	$2K$
`full`	$(C_i, Y_{i,1}, \dots, Y_{i,K}, N_{i,1}, \dots, N_{i,K})$	$1 + 2K$

Let $\tau \in \{S, H\}$ denote shared or horizon-wise coefficients. For any coefficient vector c, set $c_h^S = c$ for every h, whereas $c_1^H, \dots, c_H^H$ are fitted independently. Define the simplex $\mathcal{S}_p = \{\lambda \in \mathbb{R}^{p+1} : \lambda_m \geq 0, \sum_{m=0}^p \lambda_m = 1\}$. The three families are

$$\widehat{G}^{R,\tau}_{i,h} = V_{i,h} + a_h^\tau V_{i,h} + \sum_{m=1}^p b^\tau_{h,m} Z_{i,h,m}, \qquad (a_h^\tau, b^\tau_{h,1}, \dots, b^\tau_{h,p}) \in \mathbb{R}^{p+1}, \tag{5}$$

$$\widehat{G}^{C,\tau}_{i,h} = \lambda^\tau_{h,0} V_{i,h} + \sum_{m=1}^p \lambda^\tau_{h,m} Z_{i,h,m}, \qquad \lambda_h^\tau \in \mathcal{S}_p, \tag{6}$$

$$\widehat{G}^{D,\tau}_{i,h} = V_{i,h} + \sum_{m=1}^p b^\tau_{h,m}(Z_{i,h,m} - V_{i,h}), \qquad b_h^\tau \in \mathbb{R}^p. \tag{7}$$

Vanilla is recovered by $(a, b) = 0$, $\lambda = (1, 0, \dots, 0)$, or $b = 0$. Ridge contains every convex forecast through $a = \lambda_0 - 1$ and $b_m = \lambda_m$. For `cov`, the formulas are $V + aV + bC$, $\lambda V + (1 - \lambda)C$, and $V + b(C - V)$.

Let $\mathcal{Q}_A$ be the windows in split $A \in \{T_1, T_2, T_{12}\}$, where $T_{12} = T_1 \cup T_2$. For $\tau = S$, set $\mathcal{O}_A^S = \mathcal{Q}_A \times \{1, \dots, H\}$; for the hth horizon fit, set $\mathcal{O}_A^{H,h} = \mathcal{Q}_A \times \{h\}$. Ridge and delta-ridge use correction target $t_{i,h} = G_{i,h} - V_{i,h}$ and design

$$d^R_{i,h} = (V_{i,h}, Z_{i,h,1}, \dots, Z_{i,h,p}) \in \mathbb{R}^{p+1}, \qquad d^D_{i,h} = (Z_{i,h,1} - V_{i,h}, \dots, Z_{i,h,p} - V_{i,h}) \in \mathbb{R}^p.$$

For either $d \in \mathbb{R}^q$ and an observation set $\mathcal{O}$, define

$$r_k = \max\left(\sqrt{\frac{1}{|\mathcal{O}|}\sum_{(i,h)\in\mathcal{O}} d_{i,h,k}^2}, 10^{-12}\right), \qquad \widetilde{d}_{i,h,k} = d_{i,h,k}/r_k,$$

and, for $\alpha \geq 0$,

$$\widehat{\gamma}_{\alpha,\mathcal{O}} = \arg\min_{\gamma\in\mathbb{R}^q} \frac{1}{|\mathcal{O}|}\sum_{(i,h)\in\mathcal{O}} (t_{i,h} - \widetilde{d}_{i,h}^{\top}\gamma)^2 + \alpha\|\gamma\|_2^2, \qquad \widehat{\beta}_{\alpha,\mathcal{O},k} = \widehat{\gamma}_{\alpha,\mathcal{O},k}/r_k.$$

The grid is

$$\mathcal{A} = \{0, 10^{-6}, 10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1, 10\}.$$

For each $\alpha \in \mathcal{A}$, coefficients fitted on T_1 are scored by nMSE on T_2; the minimizer (smallest α on ties) is refitted on T_{12}. Convex coefficients are fitted directly on T_{12}:

$$\widehat{\lambda}_{\mathcal{O}} = \arg\min_{\lambda\in\mathcal{S}_p} \frac{1}{|\mathcal{O}|}\sum_{(i,h)\in\mathcal{O}} \left(G_{i,h} - \lambda_0 V_{i,h} - \sum_{m=1}^{p} \lambda_m Z_{i,h,m}\right)^2.$$

Use $\mathcal{O} = \mathcal{O}_A^S$ for a shared fit and $\mathcal{O} = \mathcal{O}_A^{H,h}$ independently for each h; no intercept or centring is added. Chunked sufficient statistics give the exact objectives.

Table 6: Baseline names; substitute a stem from Table 5.

Artifact	Definition	Fitted scalars
`vanilla`	$V_{i,h}$	0
`cov_forecast`	$C_{i,h}$	0
`avgy`	$\overline{Y}_{i,h}$	0
`y_mean`	$K^{-1}\sum_j Y_{i,j,h}$	0
`<stem>_ridge_*`	Equation (5), $*$ = `shared/horizon`	$(p+1)$ / $H(p+1)$
`<stem>_convex_*`	Equation (6), $*$ = `shared/horizon`	$(p+1)$ / $H(p+1)$
`<stem>_delta_ridge_*`	Equation (7), $*$ = `shared/horizon`	p / Hp

6 Gates and features

A gate compares V_i with $A_i = C_i$ (`cov`) or $A_i = \overline{Y}_i$ (`avgy`). Define the pointwise and shared candidate advantages

$$\Delta_{i,h} = (G_{i,h} - V_{i,h})^2 - (G_{i,h} - A_{i,h})^2, \qquad \overline{\Delta}_i = H^{-1}\sum_{h=1}^{H} \Delta_{i,h}.$$

For $q = (q_1, \ldots, q_K) \in \mathbb{R}^K$, define neighbour-weighted moments

$$\mu_{w_i}(q) = \sum_{j=1}^{K} w_{i,j} q_j, \qquad \sigma_{w_i}(q) = \left[\sum_{j=1}^{K} w_{i,j}(q_j - \mu_{w_i}(q))^2\right]^{1/2}.$$

Let μ_h and σ_h denote population moments over $h \in \{1, \ldots, H\}$. For neighbour j, set

$$r_{i,j}^V = \mu_h(Y_{i,j,h} - V_{i,h}), \qquad s_{i,j}^V = \sigma_h(Y_{i,j,h} - V_{i,h}),$$
$$r_{i,j}^N = \mu_h(Y_{i,j,h} - N_{i,j,h}), \qquad s_{i,j}^N = \sigma_h(Y_{i,j,h} - N_{i,j,h}).$$

Let $(r_{i,j}, v_{i,j})$ be neighbour j's date and channel, and define

$$\boldsymbol{\ell}_i = (\mu_{X_{i,1}}, \ldots, \mu_{X_{i,K}}), \quad \boldsymbol{\rho}_i = (\sigma_{X_{i,1}}, \ldots, \sigma_{X_{i,K}}), \quad \boldsymbol{a}_i = (s_i - r_{i,1}, \ldots, s_i - r_{i,K}),$$
$$\boldsymbol{w}_i = (w_{i,1}, \ldots, w_{i,K}), \qquad \boldsymbol{d}_i = (d_{i,1}, \ldots, d_{i,K}).$$

The exact static vector $f_i^0 \in \mathbb{R}^{18}$ is

$$f_i^0 = (\mu_{w_i}(\boldsymbol{r}_i^V), \sigma_{w_i}(\boldsymbol{r}_i^V), \mu_{w_i}(\boldsymbol{s}_i^V), \mu_{w_i}(\boldsymbol{r}_i^N), \sigma_{w_i}(\boldsymbol{r}_i^N), \mu_{w_i}(\boldsymbol{s}_i^N),$$
$$\mu_{X_{s_i,u_i}}, \sigma_{X_{s_i,u_i}}, \mu_{\boldsymbol{\ell}_i}, \sigma_{\boldsymbol{\ell}_i}, \mu_{\boldsymbol{\rho}_i}, \sigma_{\boldsymbol{\rho}_i}, K^{-1}\textstyle\sum_{j=1}^{K} \mathbf{1}[v_{i,j} = u_i],$$
$$\mu_{\boldsymbol{a}_i}, \sigma_{\boldsymbol{a}_i}, \sigma_{\boldsymbol{w}_i}, \max_j w_{i,j}, \mu_{\boldsymbol{d}_i}),$$

where $\boldsymbol{r}_i^V = (r_{i,j}^V)_{j=1}^K$ and analogously for $\boldsymbol{s}_i^V, \boldsymbol{r}_i^N, \boldsymbol{s}_i^N$. The shared and horizon feature vectors are

$$f_i^S = (\mu_h(A_{i,h} - V_{i,h}), \sigma_h(A_{i,h} - V_{i,h}), f_i^0) \in \mathbb{R}^{20}, \tag{8}$$

$$f_{i,h}^H = (A_{i,h} - V_{i,h}, \mu_{w_i}((Y_{i,j,h} - V_{i,h})_j), \sigma_{w_i}((Y_{i,j,h} - V_{i,h})_j), \mu_{w_i}((Y_{i,j,h} - N_{i,j,h})_j), \sigma_{w_i}((Y_{i,j,h} - N_{i,j,h})_j), f_i^0) \in \mathbb{R}^{23}. \tag{9}$$

For shape $\tau \in \{S, H\}$, write $z_i^S = \overline{\Delta}_i$, $z_{i,h}^H = \Delta_{i,h}$, f_i^S as in Equation (8), and $f_{i,h}^H$ as in Equation (9). A shared model is fitted once; a horizon model is fitted independently for each h. CatBoost scores are

$$g^{\mathrm{reg},\tau} = \mathrm{CBReg}(f^\tau), \qquad g^{\mathrm{cls},\tau} = \Pr_{\mathrm{CBCls}}(z^\tau > 0 \mid f^\tau) - \tfrac{1}{2}.$$

The tree count is selected on T_2 after fitting on T_1, then the model is refitted on T_{12} with that count. Define

$$q^{\mathrm{hard},\tau} = \mathbf{1}[g^\tau > 0], \qquad \widehat{G}_{i,h}^{\mathrm{hard},\tau} = (1 - q_{i,h}^{\mathrm{hard},\tau})V_{i,h} + q_{i,h}^{\mathrm{hard},\tau} A_{i,h},$$

with $q_{i,h}^{\cdot,S} = q_i^{\cdot,S}$. For soft output, let $\mathrm{sigmoid}(x) = (1 + e^{-x})^{-1}$ and

$$s^S = \mathrm{sd}((z_i^S)_{i \in \mathcal{Q}_{T_{12}}}), \qquad s_h^H = \mathrm{sd}((z_{i,h}^H)_{i \in \mathcal{Q}_{T_{12}}}).$$

Writing s^τ for the matching shared or hth-horizon scale,

$$p^{\mathrm{cls},\tau} = g^{\mathrm{cls},\tau} + \tfrac{1}{2}, \qquad p^{\mathrm{reg},\tau} = \begin{cases} \mathrm{sigmoid}(g^{\mathrm{reg},\tau}/s^\tau), & s^\tau > 10^{-12}, \\ 0, & s^\tau \le 10^{-12},\ g^{\mathrm{reg},\tau} < 0, \\ \frac{1}{2}, & s^\tau \le 10^{-12},\ g^{\mathrm{reg},\tau} = 0, \\ 1, & s^\tau \le 10^{-12},\ g^{\mathrm{reg},\tau} > 0, \end{cases}$$

$$\widehat{G}_{i,h}^{\mathrm{soft},\tau} = (1 - p_{i,h}^\tau)V_{i,h} + p_{i,h}^\tau A_{i,h}.$$

For shared output, $p_{i,h}^{\cdot,S} = p_i^{\cdot,S}$ for every h. The no-feature Bayes scores and scored-split oracle scores are

$$g^{B,S} = |\mathcal{Q}_{T_{12}}|^{-1} \sum_{i \in \mathcal{Q}_{T_{12}}} \overline{\Delta}_i, \quad g_h^{B,H} = |\mathcal{Q}_{T_{12}}|^{-1} \sum_{i \in \mathcal{Q}_{T_{12}}} \Delta_{i,h}, \quad g_i^{O,S} = \overline{\Delta}_i, \quad g_{i,h}^{O,H} = \Delta_{i,h},$$

and use the hard rule above.

Table 7: Gate names for $A \in \{\texttt{cov}, \texttt{avgy}\}$; $\tau = S/H$.

Name pattern	Definition	Fitted scalars / proxy
`bayes_<A>_shared/horizon`	$g^{B,\tau}$, hard	1 / H
`catboost_<A>_regressor_shared/horizon`	$g^{\mathrm{reg},\tau}$, hard	≤ 6000 / $\le 6000H$
`catboost_<A>_classifier_shared/horizon`	$g^{\mathrm{cls},\tau}$, hard	≤ 6000 / $\le 6000H$
`catboost_<A>_*_shared/horizon_soft`	same fit, soft output	0 additional
`oracle_<A>_shared/horizon`	$g^{O,\tau}$, hard	0

CatBoost uses at most 300 depth-4 trees, learning rate 0.03, 50-round early stopping, and balanced classifier classes (Prokhorenkova et al., 2018). The proxy is $300(16 \text{ leaf} + 4 \text{ split}) = 6000$. Screen uses only hard shared regressors; classifier, soft-regressor, and horizon-regressor forms are one-axis ablations.

7 Practical protocol

7.1 Chronological regions and window eligibility

For the T dates introduced in Section 1, extraction computes exclusive boundaries, where round means nearest-integer rounding and $[a, b) = \{a, a+1, \ldots, b-1\}$ for integer dates:

$$b_0 = \mathrm{round}(0.30T), \qquad b_{12} = \mathrm{round}(0.80T), \qquad b_3 = T.$$

The extraction regions are the fixed datastore region $T_0 = [0, b_0)$, pooled adaptation region $T_{12} = [b_0, b_{12})$, and held-out evaluation region $T_3 = [b_{12}, T)$. For a target period $[b, e)$, eligible query dates are

$$s \in [\max(L - 1, b - 1),\ \min(T - H - 1, e - H - 1)]$$

at the configured query stride. Hence the first target may start at b with query index $b-1$; its lookback may cross the boundary, but its complete target must be inside $[b, e)$. Changing L therefore preserves T_3 target dates whenever both runs remain eligible.

Each downstream learner re-splits T_{12} by unique query date. If $n_{12} \geq 2$ dates are present and the validation fraction is $\rho = 0.2$,

$$n_2 = \min(n_{12} - 1, \max(1, \text{round}(\rho n_{12}))), \qquad n_1 = n_{12} - n_2.$$

The first n_1 and last n_2 whole dates form T_1 and T_2, respectively, so $T_{12} = T_1 \cup T_2$ and the two sets are disjoint; users on the same date are never split.

7.2 Datastore date grids

Let P be the retrieval period, $D_{\text{stride}} > 0$ the datastore stride, and $A \in \{0, 1\}$ the period-alignment indicator. Alignment requires $P \mid D_{\text{stride}}$. Before optional explicit history bounds, define the admissible endpoint pair

$$(a_s, b_s) = \begin{cases} (L-1, b_0 - H - 1), & \text{fixed mode,} \\ (L-1, s - H), & \text{online mode.} \end{cases}$$

The first candidate and uncapped date grid are

$$r_s^A = \begin{cases} a_s + ((s - a_s) \bmod P), & A = 1, \\ a_s, & A = 0, \end{cases} \qquad \mathcal{G}_s = \{r_s^A + \ell D_{\text{stride}} : \ell \in \mathbb{N}_0,\ r_s^A + \ell D_{\text{stride}} \leq b_s\}.$$

Let $\text{Tail}_M(\mathcal{G})$ denote the M most recent ordered elements of $\mathcal{G}$, with $\text{Tail}_{+\infty}(\mathcal{G}) = \mathcal{G}$. Let M_s be the minimum of the active date cap, the window-derived cap $\lfloor W_{\max}/U \rfloor$, and, in default online mode, the number of dates in the fixed-mode grid for the same s; an absent cap is $+\infty$, and `full_online_history` omits the last term. The code returns

$$\mathcal{R}_s = \text{Tail}_{M_s}(\mathcal{G}_s).$$

Thus $A = 1$ implies $(s - r) \bmod P = 0$ for every $r \in \mathcal{R}_s$; $A = 0$ imposes no phase constraint. In fixed mode, both $(r - L, r]$ and $(r, r + H]$ lie in T_0. In online mode,

$$r \geq L - 1, \qquad r + H \leq s,$$

so the neighbour future is already observed. It may overlap the query lookback, but never the query target.

Without an explicit override, the code first counts the dates that a fixed T_0 store would provide under the same value of A. The online store then keeps that many *most recent* eligible dates, subject to the 30,000-window cap. Thus the store slides forward without growing merely because the query is later. `full_online_history` removes this implicit date-count cap; explicit date and window caps still apply. Defaults are $P = 96$ samples for ETTm1, ETTm2, ETT_T_15T, and ETT_L_15T, and $P = 24$ otherwise. Publication profiles disable period alignment so their datastore origins rotate through seasonal phases. If period alignment is explicitly enabled, the datastore stride must be a multiple of P.

7.3 Fitting and leakage controls

Table 8: What each chronological region may influence.

Family	T_1	T_2	T_3
Convex mixtures	diagnostic fit	final fit on pooled T_{12}	score once
Ridge baselines	fit all α	select α; refit pooled	score once
Fixed-candidate CatBoost	fit up to 300 trees	select tree count; fresh pooled refit	score once
TS-IFA joint	optimize branches and one rooter	select and restore checkpoint	score once
TS-IFA meta	chronological support/query outer updates	fit active frozen-branch rooter	score once
Oracle diagnostics	–	–	target-aware appendix bound only

Ridge sufficient statistics are accumulated exactly in float64 chunks. Optional sample caps affect fitting only; final T_3 scoring always uses every saved sample. A future gate over a trainable candidate must train that candidate on T_1 and the gate on T_2 without pooled refitting, unless out-of-fold candidate predictions are introduced.

Dataset loaders discover `config.json` beside the selected CSV. Portable fields live at the JSON top level and adaptation-only overrides under `adaptation`. Project-scoped values override shared values, explicit run arguments override both, and `drop_users` is additive. The resolved path and applied keys are recorded in the extraction manifest; no file content hash is created.

8 TS-IFA

TS-IFA is a lightweight neural adapter around the frozen backbone $F := F_\theta$. We again suppress (s, u), so $X = X_{s,u}$ and $Y = Y_{s,u}$; all following values are on the query scale:

$$p = F(X), \qquad p^c = F(X; \{X_j, Y_j\}_{j=1}^K), \qquad n_j = F(X_j), \qquad e_j = Y_j - n_j.$$

8.1 Candidate branches

Define the learned maps

$$\begin{aligned} A_r &: \mathbb{R}^{L+H} \times (\mathbb{R}^{L+H})^K \times (\mathbb{R}^H)^K \to \mathbb{R}^H, & M_r &: \mathbb{R}^{2H} \to \mathbb{R}^H, \\ A_m &: \mathbb{R}^{L} \times (\mathbb{R}^{L})^K \times (\mathbb{R}^H)^K \to \mathbb{R}^H, & M_m &: \mathbb{R}^{2H} \to \mathbb{R}^H. \end{aligned}$$

A_r, A_m are multi-head cross-attention maps and M_r, M_m are correction MLPs. Residual attention compares query and neighbour *states*:

$$q_r = [X, p], \qquad k_j^r = [X_j, n_j], \qquad v_j^r = e_j, \tag{10}$$

$$z_r = A_r(q_r, \{k_j^r\}, \{v_j^r\}), \qquad c_r = p + M_r([p, z_r]). \tag{11}$$

Memory attention reads realised neighbour horizons directly:

$$z_m = A_m(X, \{X_j\}, \{Y_j\}), \qquad c_m = p + M_m([p, z_m]).$$

Vanilla is always present, and any non-empty subset of $\{p^c, c_r, c_m\}$ can be selected. For active candidate set I, the unconstrained rooters operate on the non-vanilla deltas $d_j = c_j - p$, $j \in I$. Fixed and learned transformed covariates are a separate experiment rather than TS-IFA candidates.

8.2 Neural and ridge rooters

The neural rooter receives no handcrafted distances or dispersion features. Its learned attention map is $A_g : \mathbb{R}^L \times (\mathbb{R}^L)^K \times (\mathbb{R}^L)^K \to \mathbb{R}^{d_g}$, where d_g is the configured representation width:

$$z_g = A_g(X, \{X_j\}, \{X_j\}) \in \mathbb{R}^{d_g}.$$

For the learned type embedding $\tau_j \in \mathbb{R}^{d_g}$ of active delta d_j, let LN denote dimension-preserving layer normalization and let $S_\omega : \mathbb{R}^{2d_g+2H} \to \mathbb{R}^R$ be the shared MLP scorer, where $R = 1$ for shared routing and $R = H$ for horizon routing. It returns

$$a_j = S_\omega([\mathrm{LN}(z_g), \mathrm{LN}(p), \mathrm{LN}(d_j), \mathrm{LN}(\tau_j)]) \in \mathbb{R}^R, \tag{12}$$

$$\widehat{Y}_{\text{neural}} = p + \sum_{j \in I} a_j \odot d_j. \tag{13}$$

Here ω denotes all neural-rooter parameters: A_g, its layer normalizations, the type embeddings, and the shared scorer. Unconstrained routing uses these signed delta weights. Optional softmax routing prepends a vanilla logit, then normalizes vanilla and every active candidate to non-negative weights summing to one. The shared form broadcasts one value over all horizons; the horizon form retains one value per horizon index.

The ridge rooter uses the same active candidates. In the joint variant its free routing matrix is $\nu \in \mathbb{R}^{|I| \times R}$ and is updated by gradient descent with the branches. Only the meta variant uses the following closed-form solve. For any

non-empty fit set $\mathcal{A}$, let $d_{i,j,h}$ be the horizon-h delta of candidate j on window i. Let $n_{\mathcal{A}} = |\mathcal{A}| \geq 1$. The differentiable meta-solve uses $\epsilon_{\text{meta}} = 10^{-8}$ and the final fit uses $\epsilon_{\text{fit}} = 10^{-12}$:

$$s_{j,h}^{(d,\mathcal{A})} = \begin{cases} \sqrt{n_{\mathcal{A}}^{-1} \sum_{i \in \mathcal{A}} d_{i,j,h}^2 + \epsilon_{\text{meta}}^2}, & \text{differentiable meta-solve,} \\ \max\left(\sqrt{n_{\mathcal{A}}^{-1} \sum_{i \in \mathcal{A}} d_{i,j,h}^2}, \epsilon_{\text{fit}}\right), & \text{final fit,} \end{cases} \qquad \widetilde{d}_{i,j,h} = d_{i,j,h} / s_{j,h}^{(d,\mathcal{A})}.$$

For ridge penalty $\alpha > 0$, the standardized coefficient matrix is

$$\Gamma_{\mathcal{A}}^*(\theta_b) = \arg\min_{\Gamma \in \mathbb{R}^{3 \times H}} \sum_{i \in \mathcal{A}} \sum_{h=1}^{H} \left(Y_{i,h} - p_{i,h} - \sum_{j=1}^{3} \Gamma_{j,h} \widetilde{d}_{i,j,h} \right)^2 + \alpha \|\Gamma\|_F^2, \qquad \nu_{\mathcal{A},j,h}^*(\theta_b) = \Gamma_{\mathcal{A},j,h}^*(\theta_b) / s_{j,h}^{(d,\mathcal{A})},$$

where $\| \cdot \|_F$ is the Frobenius norm. This exact solve applies to unconstrained meta-ridge; softmax meta-ridge uses differentiable gradient inner updates. The unconstrained ridge prediction is $p_{i,h} + \sum_{j=1}^{3} \nu_{j,h} d_{i,j,h}$. Both ridge variants have exactly $|I|R$ routing parameters; $\alpha = 10^{-2}$ applies to exact meta-ridge solves only.

8.3 Training and initialization

Let θ_b collect the parameters of c_r and c_m, and let $\mathcal{B} = \{c_r, c_m\}$. For minibatch index set $\mathcal{A}$, active-rooter parameters $\psi \in \{\nu, \omega\}$, and coefficients $a_{i,j,h}$, define

$$\mathcal{L}_B^{\mathcal{A}}(\theta_b) = \sum_{c \in \mathcal{B}} \text{nMSE}_{\mathcal{A}}(c, Y) + \lambda_V \sum_{c \in \mathcal{B}} \text{nMSE}_{\mathcal{A}}(c, p), \tag{14}$$

$$\mathcal{L}_R^{\mathcal{A}}(\theta_b, \psi) = \text{nMSE}_{\mathcal{A}}(\widehat{Y}, Y) + \lambda_V \, \text{nMSE}_{\mathcal{A}}(\widehat{Y}, p) + \frac{\lambda_2}{3|\mathcal{A}|H} \sum_{i,j,h} a_{i,j,h}^2 + \frac{\lambda_S}{3|\mathcal{A}|(H-1)} \sum_{i,j,h<H} (a_{i,j,h+1} - a_{i,j,h})^2. \tag{15}$$

The last term is zero for $H = 1$. Active values are $\lambda_V = \lambda_2 = \lambda_S = 10^{-2}$ and $\lambda_B = 0.1$.

The two joint variants use all of T_1:

$$(\theta_b^*, \nu^*) = \arg\min_{\theta_b, \nu} \mathcal{L}_R^{T_1}(\theta_b, \nu) + \lambda_B \mathcal{L}_B^{T_1}(\theta_b), \qquad \text{joint ridge,} \tag{16}$$

$$(\theta_b^*, \omega^*) = \arg\min_{\theta_b, \omega} \mathcal{L}_R^{T_1}(\theta_b, \omega) + \lambda_B \mathcal{L}_B^{T_1}(\theta_b), \qquad \text{joint neural.} \tag{17}$$

AdamW updates both parameter blocks. Joint ridge never uses a closed-form solve. The minimum-nMSE complete checkpoint on T_2 is restored, then scored once on T_3; no joint rooter is refitted on T_2.

For meta learning, split T_1 by whole query dates into earlier support $\mathcal{S}_1$ and later query $\mathcal{Q}_1$. Meta ridge computes

$$\nu_e^*(\theta_b) = \nu_{\mathcal{S}_1^{(e)}}^*(\theta_b), \tag{18}$$

$$\theta_b^* = \arg\min_{\theta_b} \mathcal{L}_R^{\mathcal{Q}_1^{(e)}}(\theta_b, \nu_e^*(\theta_b)) + \lambda_B \mathcal{L}_B^{\mathcal{Q}_1^{(e)}}(\theta_b), \tag{19}$$

differentiating through each exact standardized $|I| \times |I|$ solve. It freezes θ_b^* and fits $\nu_{T_2}^*(\theta_b^*)$ in closed form. Meta neural starts from learned $\omega^{(0)} = \omega$ and takes

$$\omega^{(k+1)} = \omega^{(k)} - \eta_{\text{in}} \nabla_{\omega^{(k)}} \mathcal{L}_R^{\mathcal{S}_1^{(e)}}(\theta_b, \omega^{(k)}).$$

Its query loss updates θ_b and the initialization ω; first-order MAML is the default. It then freezes θ_b and fits only ω on all T_2. Both meta variants score T_3 once. The default query fraction is 0.2, $K_{\text{in}} = 1$, and $\eta_{\text{in}} = 10^{-3}$.

Output-zero initialization of M_r, M_m, the neural scorer, and the free joint-ridge matrix gives $c_r = c_m = p$, $a_j = 0$, and $\widehat{Y} = p$ at step zero. All AdamW stages use learning rate 10^{-5}, weight decay 10^{-4}, batch size 256, and 20,000 one-random-batch epochs outside smoke mode. The exact variant, split, optimizer, regularizers, widths, caps, and seed enter the completion signature.

8.4 Parameter counts

The full horizon-routing reference configuration uses four attention heads, per-head dimension 32, 128-unit hidden layers, and three rooter deltas. Shared routing or a branch subset reduces the rooter and branch counts. K affects compute and memory, not parameter count.

Table 9: Full horizon-routing TS-IFA reference parameter counts, reproduced by the implementation and synchronized `config.json` artifacts.

L	H	Branches	Neural rooter	Full neural	Branches + ridge
168	24	522,080	323,576	845,656	522,152
336	48	645,056	397,424	1,042,480	645,200
504	168	915,872	508,616	1,424,488	916,376
512	64	759,808	471,232	1,231,040	760,000

At 512:64, the default ridge rooter has $3H = 192$ coefficients.

Table 10: Adaptation-parameter comparison at the released Cross-RAG 512:64 setting. Frozen backbone parameters are excluded from the first numeric column. External counts follow the released implementations (Ning et al., 2025b; Lee et al., 2026b).

Method	Trainable adaptation params.	× TS-IFA	Share of backbone
TS-IFA neural	1,231,040	1.00	1.03% of Chronos-2
TS-IFA branches + ridge	760,000	0.62	0.63% of Chronos-2
TS-RAG added modules	4,775,425	3.88	2.33% of Bolt-base
Cross-RAG added modules	8,712,192	7.08	4.25% of Bolt-base
Full horizon ridge baseline, $K = 3$	512	0.00042	0.00025% of Bolt-base
One shared CatBoost gate	$\leq 6,000$ proxy	≤ 0.0049	≤ 0.005% of Chronos-2

9 Experimental design

9.1 Datasets and model profiles

Table 11: Dataset panels and their roles. All non-date variates are retained for ETT, Weather, and Exchange.

Role	Datasets	Preparation
Primary screen	Electricity, Traffic, Solar, exchange_rate	Hourly Electricity/Traffic; Solar is summed from 10-minute to hourly; Exchange is daily. Electricity drops the nine configured channels.
Full homogeneous panels	ETT_T_1H, ETT_L_1H, ETT_T_15T, ETT_L_15T	Temperature and load are separated. Hourly panels use $P = 24$; 15-minute panels use $P = 96$.
Mixed-quantity ablation	ETTh1, ETTh2, ETTm1, ETTm2, Weather	Original seven-variable ETT panels; Weather is hourly mean aggregation. Cross-variable retrieval is reported only as an ablation.

Table 12: Resolved experiment profiles. A selected winner name fixes family, formula, retrieval space, metric, K, and online mode.

Profile	Datasets	(L, H)	Backbone	Retrieval / methods
`test`	Electricity	504:168	Chronos-2	raw Euclidean, $K = 3$; smoke-only
`screen`	four primary	168:24, 336:48, 504:168	Chronos-2	raw/instance Euclidean, $K \in \{1, 3\}$; 10 baselines and 8 gate/reference artifacts
`full`	primary + four separated ETT	default three	Chronos-2	selected complete screen pipelines only
`ultra`	same as full	default three	Chronos-2, TabPFN-TS	same winners and output root as full
`crossrag`	four primary	512:64	Chronos-2, Chronos-Bolt	winner at selected K on Chronos-2; winner and released Cross-RAG at $K = 15$ on Chronos-Bolt

Let

$$\mathcal{D}_P = \{\text{Electricity, Traffic, Solar, exchange\_rate}\}, \quad \mathcal{S} = \{168{:}24, 336{:}48, 504{:}168\},$$

$$\mathcal{R} = \{\text{raw, instance}\} \times \{\text{Euclidean}\} \times \{K = 1, 3\} \times \{\text{online}\},$$

and let $\mathcal{D}_B$ be the seven stems in Table 5. The $12 = |\mathcal{D}_P||\mathcal{S}|$ screen settings test, for every $r \in \mathcal{R}$,

$$\mathcal{B}_{\text{screen}} = \{\texttt{cov\_forecast}, \texttt{avgy}, \texttt{y\_mean}\} \cup \{\texttt{<d>\_ridge\_shared} : d \in \mathcal{D}_B\},$$

$$\mathcal{G}_{\text{screen}} = \bigcup_{A \in \{\texttt{cov},\texttt{avgy}\}} \{\texttt{direct\_<A>}, \texttt{bayes\_<A>\_shared}, \texttt{catboost\_<A>\_regressor\_shared}, \texttt{oracle\_<A>\_shared}\},$$

where `direct_cov`=`cov_forecast` and `direct_avgy`=`avgy`. Thus the baseline and gate screens contain 10 and 8 artifacts per retrieval pipeline; convex, delta, horizon, classifier, and mixture forms are excluded.

Default publication datastore, pooled-adaptation-query, and evaluation-query strides are 25/25/127, respectively, with period alignment disabled. All three strides are coprime with the 24-step hourly and 96-step quarter-hourly periods, so sampled origins rotate through every seasonal phase. The datastore cap is 30,000 windows. The test profile uses aligned strides 168/256/256 and 2,048 store windows. Seed 1 is used throughout.

9.2 Benchmarks and ablations

Table 13: Selected baseline controls and one-family transformations. Retrieval metric is Euclidean and mode is online in every row.

Stem	Space, K	Horizon front	Convex front	Delta front
`full`	instance, 3	ridge shared / horizon	ridge / convex shared	ridge / delta shared
`y`	instance, 3	ridge shared / horizon	ridge / convex shared	ridge / delta shared
`cov_y`	instance, 3	ridge shared / horizon	ridge / convex shared	ridge / delta shared
`residual`	instance, 3	ridge shared / horizon	ridge / convex shared	ridge / delta shared
`cov_avgy`	raw, 3	ridge shared / horizon	ridge / convex shared	ridge / delta shared
`cov`	raw, 3	ridge shared / horizon	ridge / convex shared	ridge / delta shared

Each row retains `<stem>_ridge_shared` as its control. The three fronts replace only the suffix by `ridge_horizon`, `convex_shared`, or `delta_ridge_shared`, respectively.

Table 14: Selected CatBoost controls and one-axis transformations.

A	Space, K	Control	Separate one-axis variants
`avgy`	instance, 3	regressor shared	classifier shared; regressor shared soft; regressor horizon
`cov`	instance, 1	regressor shared	classifier shared; regressor shared soft; regressor horizon

For each A, the CatBoost front also includes matching direct, Bayes, and oracle references. No transformation is crossed with another.

Table 15: Root submission fronts and tested model sets.

Front	Model set / controlled axis
`screen.slurm`	$\mathcal{B}_{\text{screen}}$ and $\mathcal{G}_{\text{screen}}$ on $\mathcal{D}_P \times \mathcal{S} \times \mathcal{R}$
`benchmark.slurm`	complete selected screen pipelines; `full` uses Chronos-2, `ultra` adds TabPFN-TS
`k_ablation.slurm`	nine selected pipelines, $K \in \{1, 3, 5, 10, 15, 20, 100\}$ only
`h_ablation.slurm`	selected pipelines, $L = 504$, $H \in \{24, 168, 504\}$ only
`l_ablation.slurm`	selected pipelines, $H = 24$, $L \in \{24, 168, 504\}$ only
`horizon_baselines_ablation.slurm`	six controls in Table 13; shared versus horizon ridge
`convex_baselines_ablation.slurm`	same six; shared ridge versus shared convex
`delta_baselines_ablation.slurm`	same six; shared ridge versus shared delta-ridge
`catboost_ablation.slurm`	two controls in Table 14; three separate one-axis variants
`mixed_quantity_ablation.slurm`	selected pipelines transferred to the five mixed-quantity panels
`crossrag.slurm`	winner on Chronos-2 at its own K; winner and Cross-RAG on Chronos-Bolt at $(L, H, K) = (512, 64, 15)$
`ts_ifa.slurm`	complete non-meta routing/branch grid over the selected test/full/ultra subset
`ts_ifa\_h\_ablation.slurm`	selected candidates at $L = 504$ over three horizons
`ts_ifa\_l\_ablation.slurm`	selected candidates at $H = 24$ over three lookbacks
`ts_ifa\_meta\_ridge.slurm`	meta-learning on textually selected ridge candidates
`ts_ifa\_meta\_neural.slurm`	meta-learning on textually selected neural candidates
`extraction.slurm`	standalone extraction smoke or TS-IFA-input front
`baselines.slurm`	standalone primary-baseline smoke front
`gates.slurm`	standalone primary-gate smoke front
`tables.slurm`	standalone result-table front

Earlier screen analysis selected six shared-ridge baselines, two hard shared CatBoost regressors, and one shared Bayes gate; no convex or delta baseline was eligible for that screen. Those historical synchronized artifacts lack enough payload and configuration evidence for current-schema reuse and are archived. They remain analysis evidence, but a current table or continued K study must rerun the affected extraction and result identities. Publication CatBoost fits allow 300 trees; only the smoke profile uses two.

9.3 Result identity, repetitions, and selection

Extraction is a reusable workflow with identity

`outputs/extraction/dataset/L_H/backbone/space/metric/k/mode/run_n/`

and vanilla uses `none/none/0/none`. Baselines, gates, Cross-RAG, and each ablation use a separate family root with identity

`outputs/adaptation/family/dataset/L_H/backbone/formula/space/metric/k/mode/run_n/`

TS-IFA instead orders `variant/routing_scope/routing_constraint/branch_set` before `space/metric/k/mode`. Each config owns one directory. Optional row and column config subsets affect only table layout.

Splits, budgets, optimizers, regularizers, fit caps, candidate lists, and evaluation controls are pipeline configs in `manifest.json`; device and scheduler placement are runtime-only. The recorded seed fixes all stochastic choices. Repeated seeds, when used, are `seed_n` leaves. Mode is only a test/full/ultra path-subset selector and is never a family, phase, path field, or computation-signature field.

Every run uses `schema_version: 1`, declared configs, optional plain provenance, launch attempts, required artifacts, and status not-run/running/interrupted/completed. Run identity contains only the schema, identity/model configs, pipeline/experiment configs, and seeds. It never fingerprints or hashes code, Slurm files, data, weights, logs, outputs, checkpoints, or directories; code/data changes are manual rerun decisions. The schema changes only for a deliberate global contract break, and `new` creates another repeat with unchanged parameters. The default `overwrite_exact` collision policy skips exact completion, resumes exact interruption, and adds `run_n` for a changed pipeline. Before a non-skipped attempt becomes running, schema-owned preparation removes stale generated artifacts while preserving `manifest.json`, archived manifest history, and completed `seed_n` leaves. Computation entry points never delete the allocated `run_n` root. A manifest-less `run_n` is an invalid partial directory and is reclaimed by the next allocation at that index. Finished work remains `ready`; completion is recorded only by the owning Slurm workflow's final successful exit. Completed manifests are authoritative and are not followed by file hashing or synchronization checks. Within that

active workflow, later stages may select ready manifests only when their launch ID matches; independent reuse remains completed-only. Tables support distinct/latest/average pipeline configurations and selected/latest/distinct/average exact repeats. Explicit pipeline filters select configurations and must match even with one run. `SELECTED_RUNS.txt` records only automatic or pinned exact repeats, and every report records requested filters and obtained manifests. Unmigratable legacy material is isolated under `archive/legacy_pre_schema_v1_2026-08-07` and is never read by current tables.

At root-workflow startup, the launcher submits `publish.slurm` with an `afterany` dependency. After any terminal producer state, including failure, cancellation, or timeout, the publisher refreshes a handoff containing only the producer's exact stdout/stderr and launch-tagged run/report roots. It excludes `*.pt`, `*.npy`, and `*.cbm`, commits only those paths on `main`, sources `$HOME/codes/proxy.sh` using the shared two-line `$HOME/codes/.secrets/proxy.credentials`, and pushes `origin/main` without pulling or creating a pull request. A repository lock serializes the complete add/commit/proxy/push transaction and waits up to ten minutes. Set `PUBLISH_RESULTS=false` to disable it; the same script accepts `-job-id` for a manual retry.

Table construction accepts only current result manifests, keeps selected pipelines family-qualified, and requires complete dataset–setting–model coverage before writing any table. It also reads every selected current `baseline_artifacts.pt`, expands the neighbour coefficients by rank, and writes exact signed coefficient CSV files and imshow heatmaps; shared models have one row and horizon models have one row per horizon. Direct baselines have no fitted coefficients and are omitted. Complete extraction, baseline, and gate configurations are reused by default, so resubmitting a profile or ablation front rebuilds these tables and plots without refitting. For the K ablation, the same stage also overlays every candidate's average held-out nMSE improvement against K and marks the global optimum. The obsolete combined `TS-IFA` directory is not a valid input; each current TS-IFA method owns its isolated manifest and result folder.

For setting c, method m, method score $\mathrm{nMSE}_{m,c}$, and matching vanilla nMSE v_c, the reported percentage improvement is

$$I_{m,c} = 100\frac{v_c - \mathrm{nMSE}_{m,c}}{v_c}.$$

The average table reports the unweighted mean of $I_{m,c}$ over complete settings, not a percentage computed after pooling losses. If n_c is the number of held-out windows in setting c, window-level win rate is

$$W_{m,c} = 100\frac{1}{n_c}\sum_{i=1}^{n_c}\mathbf{1}\left[\frac{1}{H}\sum_{h=1}^{H}(G_{i,h} - \widehat{G}_{m,i,h})^2 < \frac{1}{H}\sum_{h=1}^{H}(G_{i,h} - V_{i,h})^2\right].$$

Final claims use complete held-out T_3 scoring. Optimistic `*_eval_fit` baselines and target-aware oracle gates are appendix diagnostics only.

10 Summary

The project tests a hierarchy of increasing adaptation capacity around frozen forecasting foundation models: direct analogue forecasts, ridge/convex/delta linear families, conditional CatBoost gates, and TS-IFA. Every method shares the same query dates, leakage-free online datastore, query-scale neighbour signals, and held-out T_3 evaluation. This makes capacity, retrieval choice, and validation protocol explicit rather than conflating them with backbone quality.

References

Amazon Science. Chronos forecasting: Official model table and code. GitHub repository, 2026. URL https://github.com/amazon-science/chronos-forecasting.

Abdul Fatir Ansari, Lorenzo Stella, Caner Turkmen, Xiyuan Zhang, Pedro Mercado, Huibin Shen, Oleksandr Shchur, Syama Sundar Rangapuram, Sebastian Pineda Arango, Shubham Kapoor, Jasper Zschiegner, Danielle C. Maddix, Hao Wang, Michael W. Mahoney, Kari Torkkola, Andrew Gordon Wilson, Michael Bohlke-Schneider, and Yuyang Wang. Chronos: Learning the language of time series. *Transactions on Machine Learning Research*, 2024. URL https://arxiv.org/abs/2403.07815.

Abdul Fatir Ansari, Oleksandr Shchur, Jaris Küken, Andreas Auer, Boran Han, Pedro Mercado, Syama Sundar Rangapuram, Huibin Shen, Lorenzo Stella, Xiyuan Zhang, Mononito Goswami, Shubham Kapoor, Danielle C. Maddix, Pablo Guerron, Tony Hu, Junming Yin, Nick Erickson, Prateek Mutalik Desai, Hao Wang, Huzefa Rangwala, George Karypis, Yuyang Wang, and Michael Bohlke-Schneider. Chronos-2: From univariate to universal forecasting. *arXiv preprint arXiv:2510.15821*, 2025. URL https://arxiv.org/abs/2510.15821.

Sungwon Han, Seungeon Lee, Meeyoung Cha, Sercan O. Arik, and Jinsung Yoon. Retrieval augmented time series forecasting. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, 2025. URL https://proceedings.mlr.press/v267/han25d.html.

Shi Bin Hoo, Samuel Müller, David Salinas, and Frank Hutter. The tabular foundation model TabPFN outperforms specialized time series forecasting models based on simple features. *arXiv preprint arXiv:2501.02945*, 2025. URL https://arxiv.org/abs/2501.02945.

Urvashi Khandelwal, Angela Fan, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Nearest neighbor machine translation. In *International Conference on Learning Representations*, 2021. URL https://arxiv.org/abs/2010.00710.

Etienne Le Naour, Tahar Nabil, and Adrien Petralia. TS-ICL: A flexible time-indexed foundation model for time series via in-context learning. *arXiv preprint arXiv:2606.05878*, 2026. URL https://arxiv.org/abs/2606.05878.

Seunghan Lee, Jaehoon Lee, Jun Seo, Sungdong Yoo, Minjae Kim, Tae Yoon Lim, Dongwan Kang, Hwanil Choi, SoonYoung Lee, and Wonbin Ahn. Cross-RAG: Zero-shot retrieval-augmented time series forecasting via cross-attention. *arXiv preprint arXiv:2603.14709*, 2026a. URL https://arxiv.org/abs/2603.14709. KDD 2026 MILETS Workshop.

Seunghan Lee et al. Cross-RAG: Official implementation. GitHub repository, 2026b. URL https://github.com/seunghan96/cross-rag.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, 2020. URL https://proceedings.neurips.cc/paper/2020/hash/6b493230-Abstract.html.

Kanghui Ning, Zijie Pan, Yu Liu, Yushan Jiang, James Yiming Zhang, Kashif Rasul, Anderson Schneider, Lintao Ma, Yuriy Nevmyvaka, and Dongjin Song. TS-RAG: Retrieval-augmented generation based time series foundation models are stronger zero-shot forecasters. In *Advances in Neural Information Processing Systems*, 2025a. URL https://arxiv.org/abs/2503.07649.

Kanghui Ning et al. TS-RAG: Official implementation. GitHub repository, 2025b. URL https://github.com/UConn-DSIS/TS-RAG.

Liudmila Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. CatBoost: Unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems*, volume 31, 2018. URL https://proceedings.neurips.cc/paper/2018/hash/14491b756b3a51daac41c24863285549-Abstract.html.

Kutay Tire, Ege Onur Taga, Muhammed Emrullah Ildiz, and Samet Oymak. Retrieval augmented time series forecasting. *arXiv preprint arXiv:2411.08249*, 2024. URL https://arxiv.org/abs/2411.08249. Revised 2026.

Silin Yang, Dong Wang, Haoqi Zheng, and Ruochun Jin. TimeRAG: Boosting LLM time series forecasting via retrieval-augmented generation. In *IEEE International Conference on Acoustics, Speech and Signal Processing*, 2025. URL https://arxiv.org/abs/2412.16643.